AI FOR DATA ANALYSIS · LESSON 4 OF 4

The AI-Assisted Analyst Workflow, Agents & Governance (Capstone)

Combine everything into one repeatable workflow: frame a question, brief the AI, analyse with code, verify, visualise and write the story. See how an analyst agent works (plan, tools, observe, repeat) with a runnable toy example, learn MCP and tool-calling basics, and finish with governance, ethics and a telecom-churn capstone.

DVA with AI · wrap-up | ✦ 2026 | Intermediate | ⏱ 140 min

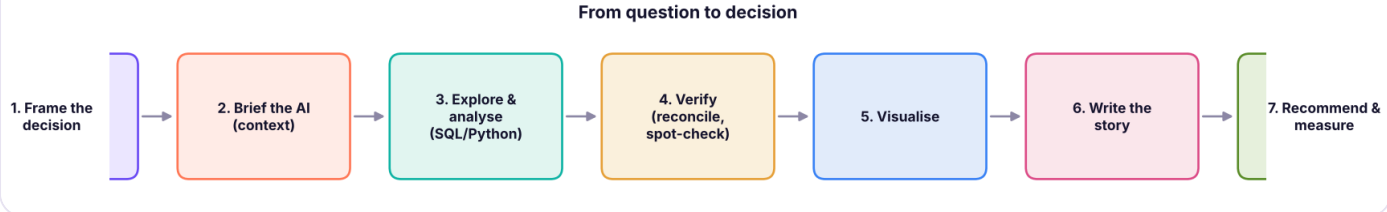
By the end of this lesson you can

- Apply a seven-step AI-assisted analysis workflow from question to recommendation
- Explain the agent loop, tool calling and the Model Context Protocol (MCP) at a conceptual level
- Build a tiny agent-style loop that plans, runs SQL tools and reports, and describe where an LLM would sit
- Design human-in-the-loop controls, logging and evaluation for AI analytics
- Complete an end-to-end churn analysis and explain it to a business audience

Datasets used

`telecom_churn.csv` `superstore_sales.csv`

The AI-assisted analyst workflow



Seven steps. AI accelerates several of them, but the analyst owns the first and the last two.

Step	What you do	Where AI helps	Your responsibility
1. Frame	Decision, audience, metric definitions, constraints	Brainstorm questions, critique your framing	Own the decision question
2. Brief	Schema, definitions, sample rows, rules	Turn notes into a clean brief	No sensitive data; correct definitions
3. Analyse	Profile, clean, aggregate, model	Draft SQL/Python, explain code, suggest checks	Run it; read it
4. Verify	Reconcile totals, test edge cases, second method	Write test queries, find anomalies	Sign off the numbers
5. Visualise	Choose and label charts	Suggest chart types, write plotting code	Honest, readable charts
6. Write	Insight, caveat, action	Draft commentary, simplify language	Every claim traceable to a number
7. Recommend	Action, owner, metric, date	Pre-mortem: "how could this fail?"	Business judgement

Case: why do telecom customers leave?

CASE STUDY **Brief**

A telecom company has 3,000 subscribers in `telecom_churn.csv` (tenure, contract type, internet service, payment method, charges, support calls, churn Yes/No). The retention head asks: **Who is most likely to churn, and what should we do about it?** We will follow the workflow and use AI-style assistance where it helps, checking everything.

Steps 1–2: frame and brief

Text

```
Decision: choose which customers get a retention offer next quarter (budget: 300 offers).
Metric: churn rate = churners / customers in the group. Report groups with at least 50
customers only.
Schema: telecom_churn(customer_id, senior_citizen 0/1, tenure_months, contract {Month-to-
month, One year, Two year},
        internet_service {Fiber, DSL, No internet}, payment_method, monthly_charges,
total_charges, support_calls, churn {Yes, No})
Rules: use only this table; state assumptions; do not claim causes; show code.
```

Step 3: profile and analyse

Python

```
import pandas as pd, numpy as np
tc = pd.read_csv("telecom_churn.csv")
tc["churned"] = (tc.churn == "Yes").astype(int)
print(f"{len(tc):,} customers | overall churn {tc.churned.mean():.1%} | missing values:
{int(tc.isna().sum().sum())} | duplicate IDs: {int(tc.customer_id.duplicated().sum())}")
tc.describe(include="all").T[["count", "unique", "top", "mean", "min",
"max"]].round(1).head(10)
```

OUTPUT

```
3,000 customers | overall churn 25.7% | missing values: 0 | duplicate IDs: 0
```

	count	unique	top	mean	min	max
customer_id	3000	3000	T00001	NaN	NaN	NaN
senior_citizen	3,000	NaN	NaN	0.17	0	1
tenure_months	3,000	NaN	NaN	36.06	0	72
contract	3000	3	Month-to-month	NaN	NaN	NaN
internet_service	3000	3	Fiber	NaN	NaN	NaN
payment_method	3000	4	Card	NaN	NaN	NaN
monthly_charges	3,000	NaN	NaN	67.07	20	131.74
total_charges	3,000	NaN	NaN	2,418.32	19.58	8,724.35
support_calls	3,000	NaN	NaN	1.28	0	6
churn	3000	2	No	NaN	NaN	NaN

10 rows × 6 columns

Churn by one factor at a time (the same GROUP BY thinking as SQL Lesson 7):

Python

```
def churn_by(col, bins=None, min_n=50):
    x = tc[col] if bins is None else pd.cut(tc[col], bins)
    g = tc.groupby(x, observed=True).agg(customers=("churned", "size"), churners=
("churned", "sum"))
    g["churn_%"] = (g.churners / g.customers * 100).round(1)
    return g[g.customers ⩾ min_n]
print(churn_by("contract").to_string()); print();
print(churn_by("internet_service").to_string()); print()
print(churn_by("support_calls").to_string()); print(); print(churn_by("tenure_months",
bins=[-1, 6, 12, 24, 48, 72]).to_string())
```

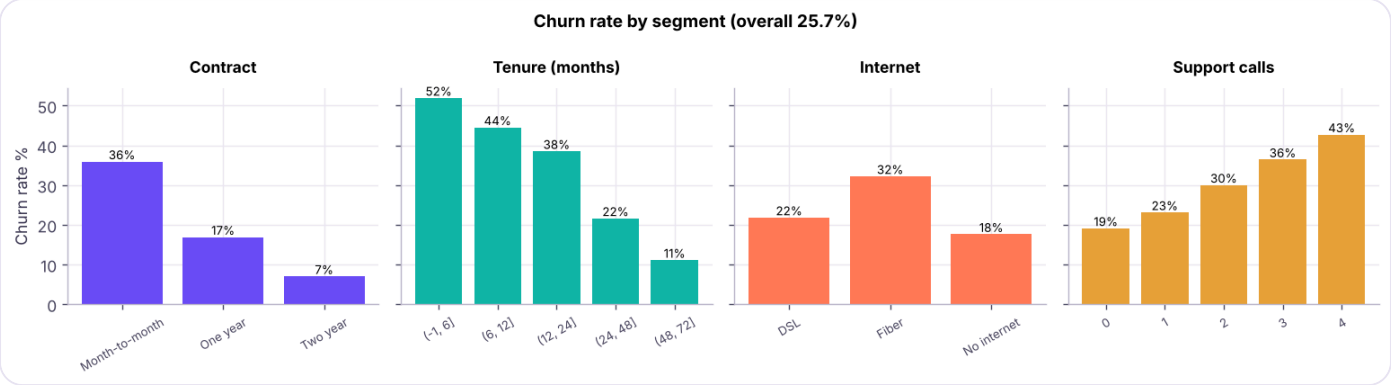
OUTPUT

```
                customers  churners  churn_%
contract
Month-to-month       1690       607     35.9
One year              739       125     16.9
Two year              571        40      7.0

                  customers  churners  churn_%
internet_service
DSL                    1047       228     21.8
Fiber                  1368       441     32.2
No internet             585       103     17.6

               customers  churners  churn_%
support_calls
0                    822       157     19.1
1                   1101       254     23.1
2                    649       195     30.0
3                    294       107     36.4
4                    110        47     42.7

               customers  churners  churn_%
tenure_months
(-1, 6]              318       165     51.9
(6, 12]              238       106     44.5
(12, 24]             452       174     38.5
(24, 48]            1011       218     21.6
(48, 72]             981       109     11.1
```



Churn rate by contract type, tenure band, internet service and support calls. Month-to-month customers, new customers, fibre users and frequent callers churn most.

Step 4: verify before you believe

Python

```
# 1. Reconcile: group counts add up; 2. a second method gives the same answer; 3. check small groups
g = churn_by("contract", min_n=0)
print("Group customers sum to total :", g.customers.sum() == len(tc))
print("Group churners sum to total  :", g.churners.sum() == tc.churned.sum())
import duckdb
sql = duckdb.sql("SELECT contract, COUNT(*) n, ROUND(100.0*AVG(CASE WHEN churn='Yes' THEN 1 ELSE 0 END),1) churn_pct FROM tc GROUP BY contract ORDER BY contract").df()
print("SQL cross-check:\n", sql.to_string(index=False))
print("\nCombinations with fewer than 50 customers are excluded from rankings; check:", (tc.groupby(['contract','internet_service']).size() < 50).sum(), "such groups")
```

OUTPUT

```
Group customers sum to total : True
Group churners sum to total  : True
SQL cross-check:
       contract    n  churn_pct
Month-to-month 1690       35.9
      One year  739       16.9
      Two year  571        7.0

Combinations with fewer than 50 customers are excluded from rankings; check: 0 such groups
```

Associations are **not causes**: month-to-month customers may also be newer or on fibre. To look at several factors together, combine them:

Python

```
tc["tenure_band"] = pd.cut(tc.tenure_months, [-1, 12, 72], labels=["≤12 months", ">12 months"])
combo = tc.groupby(["contract", "tenure_band"], observed=True).agg(customers=("churned", "size"), churn_pct=("churned", lambda s: round(s.mean() * 100, 1)))
combo[combo.customers ⩾ 50]
```

OUTPUT

		customers	churn_pct
contract	**tenure_band**		
Month-to-month	**≤12 months**	306	67.0
	>12 months	1384	29.0
One year	**≤12 months**	151	33.8
	>12 months	588	12.6
Two year	**≤12 months**	99	15.2
	>12 months	472	5.3

6 rows × 2 columns

Steps 5–7: prioritise and recommend

With a budget of 300 offers, focus on the **highest-risk group that is also large enough to matter**. A simple **risk × size** table:

Python

```
seg = tc.groupby(["contract", "internet_service"], observed=True).agg(customers=("churned", "size"), churn_pct=("churned", "mean"), avg_charges=("monthly_charges", "mean"))
seg["expected_churners"] = (seg.customers * seg.churn_pct).round(0)
seg["churn_pct"] = (seg.churn_pct * 100).round(1); seg["avg_charges"] = seg.avg_charges.round(0)
seg = seg[seg.customers ⩾ 50].sort_values("expected_churners", ascending=False)
seg.head(6)
```

OUTPUT

		customers	churn_pct	avg_charges	expected_churn
contract	**internet_service**				
Month-to-month	**Fiber**	743	43.5	87	323
	DSL	617	30.8	61	190
	No internet	330	28.5	31	94
One year	**Fiber**	341	26.7	86	91
	DSL	252	11.1	61	28
Two year	**Fiber**	284	9.5	87	27

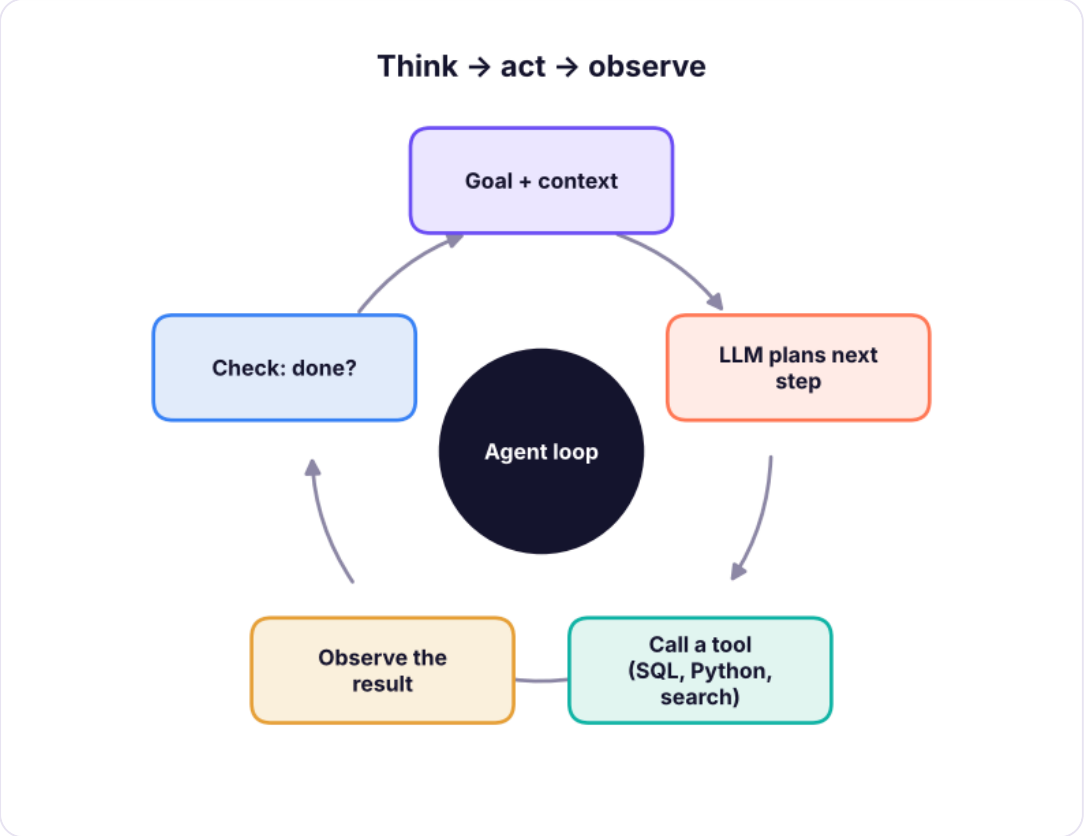
6 rows × 4 columns

KEY TAKEAWAYS **A recommendation a manager can act on**

1. **Target** the top segments by *expected churners*, not only by churn rate: they hold most of the preventable loss.
2. **Offer** a discounted switch to a 1-year contract for month-to-month customers in their first year, and a service check for fibre customers with 2+ support calls.
3. **Measure** with an A/B test: random half of the target list gets the offer; compare churn after 90 days (the test design is in the ML track, hypothesis-testing lessons).
4. **Limit**: this analysis is descriptive; it does not prove that contracts *cause* churn or estimate the offer's effect.

How an AI agent does analysis

An agent turns the workflow above into a loop that an LLM drives:



The agent loop. The model decides the next action, a tool executes it, and the result goes back into the model's context.

Piece	Meaning	Analytics example
Goal	What to achieve	"Find the top churn drivers and draft a memo"
Tools	Functions the model can call	`run_sql(query)`, `run_python(code)`, `plot(spec)`, `search_docs(text)`
Memory / context	What it has seen so far	Previous queries and results
Planner (LLM)	Chooses the next action	"Check churn by contract next"
Guardrails	Limits	Read-only DB, max queries, no network, human approval for writes
Evaluation	How you know it worked	Unit tests on metrics, golden questions, human review

A toy agent you can run (no LLM)

To make the loop concrete, here is a miniature "analyst agent" in which a hard-coded planner stands in for the LLM. It has **one tool** (read-only SQL) and a rule: stop when it has found the top three churn drivers.

Python

```python
import duckdb
db = duckdb.connect(); db.execute("CREATE TABLE tc AS SELECT * FROM
read_csv_auto('telecom_churn.csv')")

def run_sql(q):                                  # TOOL: read-only SQL with a row limit
    assert q.strip().lower().startswith("select"), "only SELECT statements are allowed"
    return db.execute(q).df().head(20)

def planner(memory):                             # stands in for the LLM: decides the next
tool call
    todo = [c for c in ["contract", "internet_service", "payment_method", "support_calls"]
if c not in memory]
    if todo and len(memory) < 4:
        c = todo[0]
        return f"SELECT {c} AS grp, COUNT(*) AS n, ROUND(100.0*AVG(CASE WHEN churn='Yes'
THEN 1 ELSE 0 END),1) AS churn_pct FROM tc GROUP BY {c} HAVING COUNT(*) ⩾ 50", c
    return None, None

memory, log = {}, []
while True:
    q, key = planner(memory)
    if q is None: break
    res = run_sql(q); memory[key] = res; log.append(f"ran query on '{key}' → best group:
{res.sort_values('churn_pct').iloc[-1].grp} ({res.churn_pct.max()}%)")
print("\n".join(log))
spread = {k: v.churn_pct.max() - v.churn_pct.min() for k, v in memory.items()}
print("\nDrivers ranked by spread in churn rate (percentage points):")
for k, v in sorted(spread.items(), key=lambda x: -x[1]): print(f"  {k:18s} {v:5.1f}")
```

OUTPUT

```
ran query on 'contract' → best group: Month-to-month (35.9%)
ran query on 'internet_service' → best group: Fiber (32.2%)
ran query on 'payment_method' → best group: Net banking (28.5%)
ran query on 'support_calls' → best group: 4.0 (42.7%)

Drivers ranked by spread in churn rate (percentage points):
  contract            28.9
  support_calls       23.6
  internet_service    14.6
  payment_method       5.0
```

A real agent replaces `planner` with an LLM call that reads the memory and writes the next SQL, adds many more tools, and decides when to stop. Everything else (read-only guard, row limit, log) is the same safety scaffolding you should demand from any agent.

Tool calling and MCP

- **Tool (function) calling**: the model returns a structured request such as `{"tool": "run_sql", "arguments": {"query": "SELECT …"}}`; your code runs it and sends the result back.
- **Model Context Protocol (MCP)** is an open standard for connecting AI applications to **tools and data sources** through a common interface, so one connector (a "server") for your database, files or SaaS app works with many AI clients. For analysts it means an assistant can query a **governed** database or semantic layer through an approved server instead of you pasting data around.
- **RAG (retrieval-augmented generation)** gives the model *documents* (metric definitions, past reports) to ground its answers (GenAI track).
- **Multi-agent systems** split work among specialised agents (a SQL agent, a chart agent, a reviewer); more moving parts mean more to test.

Evaluating AI analytics

What to evaluate	How
Correctness of numbers	A set of "golden questions" with known answers (total sales by year, churn by contract) run on every change
SQL quality	Execution accuracy (does the result match the reference result?), not just text similarity
Faithfulness of narrative	Every number in the text must appear in the tables; automatic check by extracting numbers
Safety	Attempted writes blocked, personal data not exposed, prompt-injection tests
Cost and latency	Tokens, tool calls and time per task
Usefulness	Do users make better decisions? Track adoption and feedback

Governance, ethics and your career

- **Policy**: know your organisation's approved tools, data-classification rules and retention.
- **Human in the loop** for decisions that affect people (credit, hiring, pricing, health): AI assists; accountable humans decide.
- **Fairness**: test whether results differ unfairly across groups (gender, age, region) before acting.
- **Explainability**: record the data, prompt, code and checks behind each insight; be able to re-run it.
- **Skills that stay valuable**: asking the right question, domain knowledge, data modelling, SQL/Python fluency to *read and verify* AI output, statistics and experimentation, storytelling and stakeholder management. The analysts who thrive are those who use AI for speed **and** can prove their answers.

2026 UPDATE

What is changing in 2026

Analytics agents are moving into the BI tools themselves (Copilot in Power BI/Fabric, Gemini in Looker and BigQuery, Snowflake Cortex, Databricks Genie, Metabase AI features). Vendors compete on accuracy with governed semantic layers, so expect "ask a question of certified metrics" to become a standard feature. Learn the concepts here and re-check each product's current capabilities, pricing and data-handling terms before adopting it.

Summary

KEY TAKEAWAYS

- Follow a **seven-step workflow**: frame, brief, analyse, verify, visualise, write, recommend; AI accelerates but you own framing, verification and judgement.
- Always **verify**: reconcile totals, cross-check with a second method, exclude tiny groups, and remember associations are not causes.
- An **agent** = goal + LLM planner + tools + memory + guardrails in a loop; tool calling and **MCP** connect models to tools and governed data.
- Evaluate AI analytics with golden questions, execution accuracy, faithfulness checks, safety tests and cost tracking.
- Keep humans accountable, log the work, check fairness and follow data policy.

Quick quiz

6 questions · tap an answer to check it

1. In the agent loop, which part executes a SQL query?

A The planner

B A tool

C The memory

D The prompt

Answer: B. The LLM plans; tools perform actions such as running SQL.

2. What is MCP?

A A chart type

B An open protocol for connecting AI applications to tools and data sources

C A database engine

D A prompt format

Answer: B. The Model Context Protocol standardises how AI clients use external tools and data.

3. Why report churn only for groups with at least 50 customers?

A To shorten the report

B Small groups give unstable percentages that can mislead

C Because SQL requires it

D To hide data

Answer: B. Small samples produce noisy rates.

4. Which statement about our churn findings is correct?

A Contracts cause churn

B Month-to-month customers churn more, but this is an association that needs testing before claiming cause

C Fibre causes churn

D Support calls prove dissatisfaction

Answer: B. Descriptive analysis shows patterns, not causes; use an experiment to test an intervention.

5. What is a "golden question" set?

A Questions that cost the most

B Questions with known correct answers used to test an AI analytics system after each change

C A list of dashboards

D Training data

Answer: B. They enable regression testing of answers.

6. Which control best limits damage if an agent writes a bad query?

A A longer prompt

B A read-only database role with row limits and logging

C A bigger model

D A louder warning

Answer: B. Technical guardrails beat instructions alone.

EXERCISE

Run the workflow on a new question

Using `superstore_sales.csv`, follow the seven steps for: *"Which customer segment × region combinations should sales focus on next quarter?"* Write the brief, run the analysis (use groupby or SQL), verify with two methods, produce one chart and write a five-sentence recommendation that includes one limitation.

Show solution (worked answer)

Brief: define "focus" (for example profit, not just sales; minimum 100 order lines). Analysis: group by segment and region, compute sales, profit, margin and lines; filter groups with ≥ 100 lines. Verify: group totals equal the overall totals; repeat with SQL; check one group manually. Chart: bubble or bar of profit by segment × region. Recommendation example: "Focus on [top profit segment-region] (profit ₹X, margin Y%); protect [healthy group]; reduce discounts in [low-margin group]. Limitation: three years of data, no cost-to-serve or competitor information; profits include discounts that may not repeat."

INTERVIEW CORNER

Questions you may be asked

Q: Walk me through how you use AI in your analysis workflow. I frame the decision, give the AI a clear brief with schema and definitions, use it to draft and explain SQL or Python, run everything myself, reconcile totals and cross-check with a second method, use AI for chart and narrative drafts, then I validate each number and take responsibility for the recommendation.

Q: What is an AI agent and how is it different from a chatbot? A chatbot responds once to a prompt; an agent pursues a goal across multiple steps by calling tools, observing results and deciding what to do next, so it needs stronger guardrails and evaluation.

Q: How do you stop an AI analytics tool from giving wrong numbers? Governed semantic layer, read-only access to curated data, golden-question tests, execution-based evaluation, human review of important outputs, and logging.

Q: A model says "contract type drives churn". What do you say? The data shows an association; to claim causation we would need an experiment or a causal method, controlling for tenure and other factors.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.